Chapter 1: Getting Started

Before beginning any analysis, we must first prepare the computing environment needed for the tasks ahead. This chapter introduces the basic setup required to follow the examples in the book, including installing R and RStudio, managing packages, and preparing a computer to work with large-scale textual datasets.

The setup and tools introduced here reflect one of the central challenges of this book: performing text mining at scale. Working with large historical corpora introduces practical constraints that are often invisible in traditional historical research. Data can exceed available memory, analyses may run slowly or fail, and results depend on how workflows are structured and resources are managed. Learning these methods therefore involves understanding not only how to run code, but how to work within the limits of real computing environments.

Because historical datasets vary widely in size—and because readers will be working on many different kinds of computers—this chapter also provides strategies for managing limited memory and processing power. When necessary, readers can work with smaller samples of the data to practice the methods. These sampled outputs may differ from the full results shown in the book, but they allow readers to follow the same analytical workflow and understand the techniques.

Analysts of historical data often collaborate with programmers or rely on ready-made interfaces, but working directly in a programming language provides control over data quality, analysis, and visualization. This introduction assumes no prior programming experience and introduces the R language as our primary tool. While both R and Python are widely used for text mining, R provides a dependable ecosystem of packages developed specifically for working with language data, allowing readers to focus on analysis rather than software compatibility.

The sections that follow guide readers through installing software, loading packages, and managing computational resources so that later chapters can focus on analyzing and interpreting historical text at scale.

System Requirements

Using a programming language and processing data requires some amount of computer resources. We suggest the following minimum system requirements to run the code examples in this book:

- Operating System: Windows 7 or macOS 10.12
- Available Disk Space: 6GB
- Memory: 8G

First Time Set Up for R and RStudio

Analysts of historical data often collaborate with skilled code practitioners or use out-of-the-box search interfaces to conduct their work, but working hands-on with a programming language gives the analyst direct control over every aspect of the data quality, its analysis and visualization. This introduction is designed to introduce you to the language R with no prior experience required.

Analysts of historical data sometimes use other languages than R, although at present, the languages Python and R are the most frequently consulted for text mining. In general, R is more frequently taught in economics and statistics departments, while Python is more frequently used in Computer Science. Both are legitimate

interfaces for analyzing historical data. Certain text-mining processes might be more suited to Python, for example named entity recognition, a process discussed in the **Dangerous Art of Text Mining** but not here, or the scraping of text from Twitter or websites. We urge those curious about these techniques to consult Melanie Klein's wonderful introduction to cultural analytics in Python.

In general, the learning curve on R is less steep for new comers, while giving users direct access to functional tools for dealing with language. R's packages are, at the moment of writing, more dependable for many kinds of textual analysis, thanks to the existence of certain software packages developed by the text mining community for the purpose of analyzing words and phrases in various configurations. A lively community of R users practices a high standard of peer review for software packages on R, which ensures that software updates are almost always kept in a state of compatibility with each other. This trustworthy community makes R an ideal choice for those who want to focus their primary efforts on analysis, as those learning R will not face many of the questions about version compatibility that face users of certain versions of Python.

The first step is to make sure that you have access to a computer that is set up to use the R programming language. The steps that follow explain how to install R on a Mac or PC (Windows machine).

We also recommend installing a desktop app called RStudio designed to make working with R as intuitive as possible. RStudio is a "code editor" that makes it easy to interact with the R language and your data as it undergoes transformations. A code editor is to the language R what Microsoft Word is to writing a document: there are ultimately many applications which one could use to write a document, but Microsoft word is a standard because it makes outlining, drafting, printing, and footnotes easy to manage. Just so, RStudio will be our preferred app for accessing R.

Afterwards, the R interface may alert the user of occasional updates to the software, which is important to keep up-to-date. When these updates are necessary, the R interface will provide its own instructions, and for the most part, it is sufficient to simply type whatever is indicated on the computer screen, perhaps restarting the computer if the user encounters difficulty.

If you have already installed R and RStudio (or a comparable development environment of your choosing), you should skip to section "Loading Libraries" below.

Installing the R Language

If you are using Mac go to the R project at https://cran.r-project.org/bin/macosx/. You will have two options of the R language to choose from: a R.pkg file and a R-arm64.pkg file. The R.pkg file is for Macs with Intel processors. The arm64 version is for Macs with arm processors. If you do not know the processor of your Mac, go to the apple menu and click on "About This Mac." Your computer's processor will be listed.

If you are using Windows go to the R project at https://cran.r-project.org/bin/windows/base/. Click "Download R for Windows" at the top of the page.

If you are using Linux your steps for installation may be different depending on your distribution. If you have Ubuntu, one of the most popular distributions, you can start by follow the instructions here: https://cloud.r-project.org/bin/linux/ubuntu/fullREADME.html.

Installing RStudio

Go to https://posit.co/download/rstudio-desktop/#download. You will have the option to download the correct version of RStudio for your operating system.

Running a Line of Code

The first step in learning R is understanding how to run a line of code.

Here are some exercises to learn how to run code on your computer. Try each and decide which ones feel most comfortable to you: - You can type a command directly into the Console pane and press `Enter`. - You can write it in an R script and execute it by placing your curser on the line you want to run and press `Ctrl + Enter` on Windows or Linux (or `Cmd + Enter` on a Mac). - You can also highlight one or more lines of code and click the green Run button at the top of the script editor.

When you start RStudio for the first time you will be presented with a screen divided into four parts, each of which represents one portion of R's coding environment.

The first place to look is the lower left-hand quadrant of the RStudio interface, where you'll see the word "Console." The panel in question is known as the "Console Tab Panel." A "console" is a window that allows the computer's user to send code directly to the computer. You may be familiar with consoles from other places on the computer: PC users use the Command Line Prompt – another name for the Console. Mac users use a console when they open the "Terminal" app to run command-line commands. In RStudio, the console panel is one of two ways that allow you to write R code interactively.

To see RStudio's console in action, inside RStudio's console panel, position your cursor at the prompt (">"). Type this simple line of code: `print("Hello World")` and press Enter/Return. The console will return "Hello World." You have just run your first line of code.

In RStudio you have the option of typing all of your code into the Console. However, running code line-by-line at the console makes it difficult to save, edit, and revisit. This is why RStudio provides another panel specifically for editing R scripts, the "text editor." The text editor panel is directly above the Console Tab panel in the upper left-hand quadrant of Rstudio.

The Text Editor in RStudio is to R code what Microsoft Word is to text files: it offers a software environment that makes it easy to write, save, revise, and annotate code. There are many other coding programs that would work, and coders comfortable with these programs may adapt our instructions. This textbook, however, recommends that first-time programmers write their code in RStudio by default.

To use the Text Editor in RStudio, you must first open a new file. Go to "File," "New File," and then "R Script." A new tab will open where you can draft multiple lines of code to be run later. For now, in your new R Script tab, type the same command as you typed into the Console: type `print("Hello World")`.

Next, you will want to "run" the line of code that you just typed. To run a line of code from the Text Editor, you will click the "Run" button at upper-right-hand corner of the Text Editor panel. You can also use your keyboard (try this too). Position your cursor anywhere in the line of code in the Text Editor, and hit the keys `CTRL + Enter` (on PCS) or 'COMMAND + Return' (on Macs). To see what you did, look at the Console again, which will continue to keep track of the computer's response to all lines of code sent to the computer. In the Console, R will return "Hello World."

Try to run the code now by typing the following into your Console and pressing Enter, or by highlighting it and clicking Run:

```
print("Hello World")
```

```
## [1] "Hello World"
```

This should display the phrase "Hello World" beneath your code, confirming that R executed the command.

R Versions

Before we start writing code for historical analysis, we need to make sure we have an up-to-date version of R installed in order to use the packages and functions in this book.

To check which version of R is currently running on your computer, run:

```r
R.version.string
```

```
## [1] "R version 4.5.2 (2025-10-31)"
```

If your version is more than one major release behind (e.g., anything below 4.3 while the current version is 4.4), you should update R from:

https://cran.r-project.org

Keeping R up to date helps ensure that your code runs smoothly—especially when installing or loading packages, which often require the latest versions of R to work correctly.

Installing R Packages

To run the code in this book, we will first need to install some R packages. The purpose of a package is to add new tools and functions to our R code.

Many packages can be installed directly from the Comprehensive R Archive Network (CRAN), a central repository where R packages are reviewed, stored, and distributed. To use a function from another package, we must first install the package with `install.packages()`, and then load it with `library()`. We install the packages here so the libraries load smoothly later, without extra setup or interruptions.

If RStudio prompts you to restart R while installing a package, click "Yes" to allow the restart, and then re-run the installation command so the package loads into the fresh session.

Packages only have to be installed once, but they need to be reloaded for each new R session using `library()`.

First we can install our primary data science packages that serve as the core of our data processing approach.

```r
# For core data science and visualization packages, central to this book
install.packages("tidyverse")

# For tokenizing, cleaning, and computing tf-idf
install.packages("tidytext")

# For high-performance tools to work with large data tables
install.packages("data.table")

# For a comprehensive text analysis toolkit
# (e.g., keywords-in-context, tokenization, document-feature matrices)
install.packages("quanteda")

# For handling and manipulating dates and times
# (useful for organizing historical data chronologically)
install.packages("lubridate")

# For stemming and lemmatization
# (reducing words to their dictionary or root forms for cleaner analysis)
install.packages("textstem")
```

We can also install packages that give us tools for creating more visualizations:

```r
# For creating word clouds from text data
install.packages("wordcloud")

# For formatting numbers, colors, and axes in visualizations
install.packages("scales")

# For ridge plots (density plots stacked by category)
install.packages("ggridges")

# For non-overlapping text labels in ggplot2
install.packages("ggrepel")

# For accessible and colorblind-friendly palettes for visualizations
install.packages("viridis")

# For creating interactive and aesthetically pleasing visualizations
install.packages("plotly")
```

Packages for document formatting and communicating data in tables:

```r
# The core engine for rendering R Markdown documents
install.packages("knitr")

# For creating elegantly formatted tables in HTML and PDF
install.packages("kableExtra")

# For arranging multiple tables, plots, or graphics on a page
install.packages("gridExtra")
```

Packages for webscraping and data extraction:

```r
# Tools for scraping data from websites
install.packages("rvest")

# Tools for working with and parsing XML or HTML documents
install.packages("xml2")

# Install tools for accessing and working with U.S. Department of Justice datasets
install.packages("usdoj")
```

While many packages can be installed from CRAN, many more specialty, research-oriented packages—such as those used in this book—are hosted on GitHub instead. Packages hosted exclusively on GitHub may be there because they change rapidly, support specialized workflows, or, as in our case, include larger datasets that CRAN is not designed to support.

When a package is hosted on GitHub, it can be installed in several ways. For this book, we try to make the installation of our packages as easy as possible while working on as many computers as possible. To do this we provide special installation scripts that automate downloading and installing the required files. These scripts are accessed using `source()`, which tells R to fetch and run the installation code directly from the provided URL. For example:

```r
source("https://raw.githubusercontent.com/Democracy-Lab/hansardr/master/tools/install_hansardr.R")
library(hansardr)

source("https://raw.githubusercontent.com/Democracy-Lab/tmha.data/main/tools/install_tmha.data.R")
library(tmha.data)
```

In RStudio, you can verify whether a package is already installed by checking the Packages tab in the lower-right panel. This tab displays all packages currently available in your R library. If a package appears in that list, it is already installed; if not, you can install it using `install.packages()`. You can also load an installed package by checking the box next to its name in this tab, which is equivalent to calling `library()` in your script.

```r
install.packages("text2vec")  # Install text2vec
```

```r
library("httr")
library("fs")
library("pdftools")
```

Loading Tools

Once we have installed a package, we can test the installation of any of these packages using the `library()` function like so:

```r
library(hansardr)
```

If the code returns an error such as `Error in library(hansardr) : there is no package called 'hansardr'`, it means the package has not been installed. If no error appears, the package has loaded successfully and its functions are ready to use.

Understanding Package Versions

Sometimes package version issues can cause code to fail. If a package is outdated, it may not behave as we expect. We can check the version of any installed package with:

```r
packageVersion("hansardr")
```

```
## [1] '1.0.0'
```

Just like we installed a package, we can also update a package using `install.packages()`:

```r
install.packages("tidytext")
```

Or we can update all of our packages at once. This step might take awhile to complete depending on the number of packages we have installed.

```r
update.packages()
```

Handling Resources

Readers will be working on many different kinds of computers, some with more or less memory and processing power. Large historical datasets can sometimes cause R to slow down or crash, especially on older computers. This section outlines a few strategies for managing resources.

To help ensure accessibility, all of the code in this book has been tested in multiple classroom settings. We demonstrate workflows that will run on a MacBook with 8 GB of RAM and roughly 10 GB of free hard-drive space. The examples have been designed to run within these constraints.

However, if you encounter additional issues—such as code running slowly, long wait times, or working on a machine with limited resources—we provide several strategies for adapting the workflows in this book to smaller or less powerful computers.

One simple approach to resource management is to remove objects from memory when they are no longer needed. To demonstrate this, we load the 1820s from the `hansardr` library.

```r
# This will give us access to the Hansard corpus
library(hansardr)

# Load just the debates from the 1820s
data("hansard_1820")
```

We can use `rm()` to remove dataframes we no longer need. While `rm()` removes the data from R, it does not clear the amount of computer memory the data is consuming. Instead, we can follow wih `gc()` to clear space and reclaim memory that can be used again for other computations.

```r
# Remove the debates from R
rm(hansard_1820)

# Reclaim computer memory
gc()
```

```
##           used (Mb) gc trigger (Mb) max used (Mb)
## Ncells  587694 31.4    1282424 68.5   720760 38.5
## Vcells 1125538  8.6    8388608 64.0  1974125 15.1
```

Another strategy is to work with a sample of the data rather than the full dataset. This allows you to practice the methods without loading everything into memory. For example, you might limit the dataset to a smaller set of speakers or select only a few years from the larger decade-long corpus. In the following code, we combine the Hansard debates with their associated metadata—specifically the dates on which each speech was given—so that we can filter the corpus to include only a few of the years we want to work with.

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.1     v tibble    3.3.0
## v lubridate 1.9.5     v tidyr     1.3.1
## v purrr     1.1.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
# Load just the debates from the 1820s
data("hansard_1820")

data("debate_metadata_1820")

merged_hansard_data <- hansard_1820 %>%
  left_join(debate_metadata_1820, by = "sentence_id")

# Keep only the years 1820-1822 instead of the full decade
hansard_sample <- merged_hansard_data %>%
  filter(speechdate >= as.Date("1820-01-01"),
         speechdate <= as.Date("1822-12-31"))
```

While taking a random sample of a data frame can improve performance, random sampling is generally discouraged because it may produce a distorted or incomplete representation of the historical record. In learning situations where the dataset cannot be loaded due to limited computational resources, a random sample can be used as a temporary workaround. In such cases, the resulting outputs may not be historically representative and may differ from the and visualizations presented in the book.

```r
# Randomly sample and keep only 10% of the dataframe
hansard_sample <- hansard_1850 %>%
  sample_frac(0.10)
```

Getting Help on Functions

When working in R, you will frequently encounter functions you have not used before. A simple way to learn what a function does is to open its documentation. We can read its documentation in RStudio by typing two question marks followed by the function name:

```
??function_name
```

For example, to read the documentation for the `library()` function, run:

```
??library
```

This opens a page in RStudio's Help pane that shows a description of the function and examples of how to use it.

Learning to consult documentation early will make it much easier to follow the examples in this book and to experiment on your own.

Loading Libraries

The "datasets" library is a set of quantitative data sets that are frequently used to teach code, although because this book is about text, we will only consult them once for the purpose of teaching.

```r
library(datasets)
```

When a software package is called by the command 'library', its data and commands are available for you to use with R. R keeps track of which packages have been installed and loaded in the 'Packages' tab, which is one of the tabs available in the lower right-hand quadrant of RStudio.

In the lower-right-hand quadrant of Rstudio, click on the "Packages" tab now, and you'll see a list of software packages that came pre-loaded with R. Notice that some of these packages are checked, meaning that they are already loaded and ready for use.

Scroll down until you see the package "datasets." It should have a check mark next to it – the result of your loading the package with the command above. You can uncheck it and re-check the box next to datasets: the "Package" panel offers you another way to load a software library. Make sure that the box next to "datasets" is checked before you move on.

One thing to remember: if you've perfectly typed a command that you expect to run, and R gives you an error message, one thing to check is whether all the libraries needed for a particular exercise are checked.

It is important to note that packages only need to be installed once but libraries need to be reloaded every time you start a new RStudio session.

Loading Data

Because we've turned on the package "datasets", we can next call the dataset "iris".

The command "data" tells R to load a data set from one of the available software libraries. Back in the Text Editor panel, type in the following line of code, then run it using the "Run" button or CNTRL+Enter / COMMAND+Return:

```
data(iris)
```

To see what you've done, look in the upper right-hand quadrant of RStudio, known as the "Workplace Browser" panel. By default, you will be looking at the "Environment" tab.

In the Environment Tab, you will see that there is only one data set currently listed: "iris." The Environment Panel tells us what data and variables are loaded. It is useful for checking what data is available and what it looks like. The Environment Panel lists "iris" as having 150 "obs" or "observations" of "5 variables," which is another way of saying that there is data that could be displayed in a spreadsheet of five rows and 160 columns, or 150 rows and 5 columns.

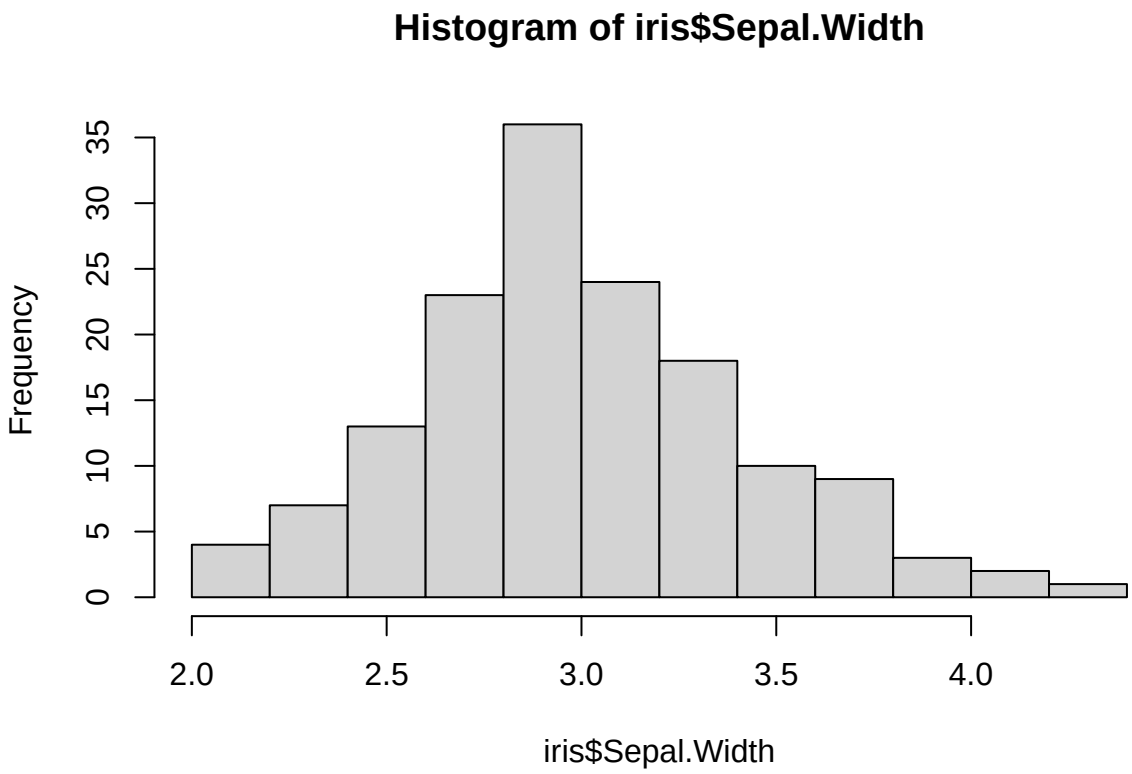
Click on the blue and white arrow button near iris to see a sample of the content of the data set.

When you run code, you may periodically want to check what your data looks like. Knowing how to inspect data in the "Environment" tab is a useful skill.

A Simple Visualization Next, we'll make a simple visualization. First, find the lower right-hand quadrant in Rstudio, where we formerly inspected software packages. Click on the "Plots" tab in this quadrant, which is where visualizations are displayed and temporarily stored. It should be blank.

Back in the Text Editor, type the following line of code and run it:

```
hist(iris$Sepal.Width)
```



The '$' marker tells R to select from the iris data set just the data in the column labeled "Sepal Width." If you're working with data that looks like a spreadsheet, for instance a data set with columns and rows, you can call the columns with the same marker. In R, these columns are known as "facets" of the data, and as we shall see, there are multiple ways of accessing them. We will only rarely see the dollar-sign marker in the future, since we will be working with another set of commands known as "tidy" commands, which make it easy to work with data that takes the form of rows and columns.

Our first visualization command, "hist," is also not one that we'll see frequently in this book, but we use it here because it's fast. The "hist" command makes a "histogram" or bar chart of frequencies for each variable in a list. In this case, the "iris" data set has a list of the width of sepals of various kinds of irises. The histogram shows us that irises with the sepal width of 3 are by far the most common of all iris kinds. Notice that R has automatically attempted to make the resulting visualization as elegant and transparent as possible, even without our providing much by way of information. The visualization has a title and its axes are labeled. In this book, we will make many more complex visualizations which have custom axis labels and captions describing the data sets' content in detail.

The "Plots" tab temporarily stores all of the visualizations you make in an R session. To store these visualizations, you can pull down the "Export" menu (just below Plots/Packages/Help/Viewer). You'll see options to export the visualization as a pdf, to the clipboard, or to another format. Choose one and save the histogram somewhere on your computer.

In Chapter 1, we will meet the "Hansard" data set which includes a century of parliamentary debates, and we will begin counting words and visualizing them.

Working with Historical Data

The data samples chosen for this book are large for an instructional textbook. One reason the samples are larger than typical is because we often work with full, decades-worth of data at a time, comparing them against one-another. The size of this data more closely resembles the kinds of data sets and computer requirements needed for serious historical research. One way we deal with the large size of our data is by designing steps that process the data in such a way as to make it small enough to run on a computer with the above mentioned minimum specifications. Even still, we encourage the reader to clear the Global Environment (and remove all saved variables) before proceeding to a new chapter. To do this, click on the broom icon in the Global Environment panel and confirm you wish to remove all objects from the environment. Next, click on the disk icon in the Global Environment panel and select "Free Unused R Memory." You may also remove specific objects from the environment using `rm()`.

```
rm(iris)
```

The Errors We All Face And How to Learn

This book is designed to be accessible to first-time coders. It assumes that you have mastered the skills of installing packages, loading packages, inspecting data, and saving visualizations described in this introduction.

One route of learning is working through the code in the chapters that follow. The reader will type each line of console-type font in to the Text Editor panel of RStudio, running one line of code from beginning to end.

In the process of typing in code, first-time programmers are likely to commit errors, and indeed even advanced programmers make typos. In programming languages, a typo will produce an error message or cause the program to perform with unintended results. For an example, try typing the following line into the R console and pressing 'enter':

```
print("hello world)
```

```
## Error in parse(text = input): <text>:1:7: unexpected INCOMPLETE_STRING
## 1: print("hello world)
##           ^
```

This line will result in an error, because a close quotation mark is missing on the other side of the phrase "hello world."

As you learn to code, whatever your level of experience, you are almost certain to have typos and errors that stop the program from performing as you expect. We advise you to ask yourself what went wrong when you inevitably get error messages or unexpected results. Through closely following the examples in the book and paying attention to any error messages that result, you will gradually learn to recognize common errors like missing parentheses or quotation marks.

Finalizing Setup

With tools installed and an environment ready, you now have everything you need to begin exploring large-scale historical text data. Every computer has limitations and each chapter will provide strategies for working effectively with the Hansard data regardless of hardware. In the following chapters, we turn from setup to analysis, beginning our journey into the methods of text mining at scale.